Encapsulation Exercise

Exercise 1: Student Information Management

Create a Java class named Student with the following private attributes:

Attribute	Data type
name	String
age	int
cgpa	double

Requirements:

1. Create a parameterized constructor to initialize the attributes.
2. Implement getter and setter methods for all attributes.
3. In setAge(), accept only values between 18 and 30.
4. In setCgpa(), accept only values between 0.0 and 4.0.
5. In setName(), reject null, empty, or whitespace-only names.
6. If invalid data is provided, display an error message and keep the previous value.
7. Create a displayInfo() method that uses getter methods.

Exercise 2: Bank Account Balance Protection

Create a Java class named BankAccount with private attributes accountNumber, accountHolder, and balance.

Requirements:

1. Initialize all attributes using a constructor.
2. Provide getter methods for all attributes.
3. Provide a setter for accountHolder with validation to reject empty names.
4. Do not provide a public setter for accountNumber or balance.
5. Create a deposit(double amount) method that accepts only positive amounts.
6. Create a withdraw(double amount) method that checks whether the amount is positive and the balance is sufficient.
7. Display the updated balance using getBalance().

Exercise 3: Employee Salary Validation

Create a Java class named Employee with the private attributes employeeId, name, and salary.

Requirements:

1. Initialize attributes using a constructor.
2. Implement getter methods for all attributes.
3. Make employeeId read-only after object creation.
4. Provide setter methods for name and salary.
5. setSalary() must reject negative values.
6. setName() must reject blank names.
7. Create a method increaseSalary(double percentage) that validates the percentage before updating the salary.

Challenge: Do not allow the employee ID to be changed after the object is created.

Exercise 4: Product Price and Stock Management

Create a class named Product containing the following private attributes: productId, productName, price, and stockQuantity.

Requirements:

1. Create a constructor to initialize all attributes.
2. Generate getters for all attributes.
3. Generate setters for productName and price.
4. A product price must be greater than zero.
5. A product name cannot be empty.
6. Do not provide a public setter for stockQuantity.
7. Create addStock(int quantity) and sellProduct(int quantity) methods.
8. Prevent selling more items than the available stock.

Sample scenario: A product has 20 units in stock. A customer purchases 5 units and then attempts to purchase 30 units.

Expected result: The second purchase is rejected, and the final stock remains 15.

Exercise 5: University Course Registration System

Create a class named Course with the following private attributes:

Attribute	Validation
courseCode	Cannot be changed after initialization
courseTitle	Cannot be empty
credit	Must be between 1 and 4
enrolledStudents	Must not be negative
maximumCapacity	Must be greater than zero

Requirements:

1. Create a parameterized constructor.
2. Provide appropriate getters.
3. Implement setters only for attributes that should be directly modifiable.
4. Implement setCredit() with range validation.
5. Implement setCourseTitle() with non-empty validation.
6. Do not provide a public setter for enrolledStudents.
7. Create enrollStudent() that increments enrollment only if space is available.
8. Create dropStudent() that decrements enrollment only if at least one student is enrolled.
9. Prevent reducing maximumCapacity below the number of currently enrolled students.

Exercise 6: Mobile Phone Battery Management

Create a class named MobilePhone containing private attributes brand, model, batteryPercentage, and isPoweredOn.

Requirements:

1. Initialize all attributes using a constructor.
2. Provide getter methods for all attributes.
3. batteryPercentage must always remain between 0 and 100.
4. Do not provide a public setter for the battery percentage.

5. Create chargeBattery(int amount) to increase battery percentage without exceeding 100.
6. Create useBattery(int amount) to decrease battery percentage without going below zero.
7. Create powerOn() and powerOff() methods.
8. A phone cannot be powered on when the battery is 0%.

Exercise 7: Online Shopping Customer Account

Create a class named Customer with private attributes customerId, name, email, phoneNumber, and walletBalance.

Requirements:

1. Initialize attributes using a constructor.
2. Provide getter methods for all attributes except where exposing data is unnecessary.
3. Do not provide a setter for customerId.
4. Validate name so that it is not blank.
5. Validate email so that it contains @ and a . after @.
6. Validate phoneNumber so that it contains exactly 11 digits.
7. Do not provide a public setter for walletBalance.
8. Create addMoney() and makePayment() methods that perform appropriate balance checks.
9. Ensure invalid setter inputs do not modify existing data.

Challenge: Prevent external code from directly modifying the wallet balance.

Exercise 8: Hospital Patient Record

Create a class named Patient with private attributes patientId, patientName, age, bloodGroup, and medicalBill.

Requirements:

1. Create a parameterized constructor.
2. Provide getters for all attributes.
3. Do not provide a setter for patientId.
4. Validate age within the range 0–120.
5. Validate bloodGroup against the accepted values: A+, A-, B+, B-, AB+, AB-, O+, and O-.

6. Allow patient name changes only when the new name is not blank.
7. Do not provide a direct public setter for medicalBill.
8. Create addMedicalCharge(double amount) to add positive charges.
9. Create makePayment(double amount) to reduce the outstanding bill without allowing overpayment.

Exercise 9: ATM PIN and Account Security

Create a class named ATMAccount containing private attributes accountNumber, pin, balance, and failedAttempts.

Requirements:

1. Initialize the account using a constructor.
2. Do not provide a getter for pin.
3. Do not provide a setter for balance.
4. Do not provide a setter for failedAttempts.
5. Create verifyPin(String enteredPin) to verify the PIN.
6. A valid PIN must contain exactly four numeric digits.
7. Create changePin(String oldPin, String newPin) to change the PIN after verifying the old PIN.
8. After three consecutive failed attempts, block further PIN verification until the account is unlocked through a separately defined administrative method.
9. Create withdraw() and deposit() methods with appropriate validation.

Challenge: Explain why giving a public getPin() method would violate secure encapsulation principles.

Exercise 10: Library Book Borrowing System

Create a class named LibraryBook containing bookId, title, author, totalCopies, and availableCopies.

Requirements:

1. Declare all attributes as private.
2. Initialize them using a constructor.
3. Provide getter methods for all attributes.

4. Do not provide setters for bookId, totalCopies, or availableCopies.
5. Allow title and author to be modified using validated setters.
6. Create borrowBook() to decrease available copies.
7. Create returnBook() to increase available copies.
8. Ensure available copies never exceed total copies and never become negative.
9. Reject borrow requests when no copies are available.

Exercise 11: Electricity Billing System

Create a class named ElectricityAccount containing private attributes meterNumber, customerName, previousReading, and currentReading.

Requirements:

1. Initialize all attributes using a constructor.
2. Make meterNumber read-only.
3. Provide getters for all attributes.
4. Validate customerName using a setter.
5. Create setCurrentReading(double reading) with validation so that the new reading cannot be lower than the previous reading.
6. Do not provide a public setter for previousReading.
7. Create calculateBill() using the following rates:

Units consumed	Rate per unit
First 100 units	5
Next 100 units	7
Above 200 units	10

8. Create startNewBillingCycle() to save the current reading as the previous reading.

Challenge: Ensure the bill cannot become negative, even when invalid input is attempted.

Exercise 12: E-Commerce Order Management

Create a class named Order containing private attributes orderId, customerName, totalAmount, status, and discountPercentage.

Requirements:

1. Initialize the attributes using a constructor.
2. Make orderId read-only.
3. Provide getter methods for all attributes.
4. Provide validated setters for customerName and discountPercentage.
5. Accept discounts only between 0% and 50%.
6. Do not provide a direct setter for status.
7. Allow only the following statuses: PLACED, SHIPPED, DELIVERED, CANCELLED.
8. Create shipOrder(), deliverOrder(), and cancelOrder() methods.
9. An order can be shipped only after being placed.
10. A shipped order can be delivered but cannot be cancelled.
11. A delivered or cancelled order cannot undergo any further status changes.
12. Create calculateFinalAmount() that returns the amount after applying the discount.

Challenge: Ensure the order status cannot be changed directly from PLACED to DELIVERED.